

Architectural Overview

The SunDevil Connect system follows a layered architectural structure based on the Boundary–Control–Entity (BCE) design model. In this structure, Boundary classes represent user interface components that interact with system users, Control classes coordinate application logic and workflow between system components, and Entity classes represent the core domain objects and persistent data of the system.

Boundary classes such as LoginPage, ClubBrowseView, and EventDetailsView manage user interaction and presentation logic. Control classes including AuthenticationController, EventBrowsingController, and ClubMembershipController coordinate system operations and enforce business rules. Entity classes such as User, Club, Event, and Announcement represent the core data structures of the application.

Within this architecture, several design patterns are applied to improve modularity, maintainability, and scalability. The Strategy Pattern and Factory Pattern are used together to support flexible event filtering behavior. The Observer Pattern is used to implement the system's notification mechanism, allowing user interfaces to react to system updates. The Singleton Pattern is applied to shared infrastructure services such as authentication, database access, and notification management to ensure consistent system-wide access.

Together, these design patterns support a modular and extensible architecture that enables SunDevil Connect to efficiently manage student clubs, events, memberships, and notifications while maintaining a clear separation of responsibilities across system components.

Design Pattern 1: Strategy Pattern

The Strategy Pattern is used in SunDevil Connect to support flexible and extensible event filtering behavior. Students can browse events and apply multiple filters such as date, category, club, or whether the event is free or paid. Instead of embedding complex conditional logic within the event browsing controller or service, the system defines a common FilterStrategy interface with a method such as applyFilter(events : List<Event>).

Concrete implementations of this interface include classes such as DateFilterStrategy, CategoryFilterStrategy, PaidFilterStrategy, and ClubFilterStrategy. Each strategy encapsulates the logic required to filter events according to a specific criterion.

When a student applies filters during event browsing, the system executes the filtering strategies sequentially on the list of events. Each strategy independently processes the event list and returns a filtered result. This modular approach allows filtering behavior to remain independent and easily extendable.

The Strategy Pattern improves system design by separating filtering algorithms from the event browsing controller, ensuring that filtering logic remains modular and maintainable. New filters can be added by simply creating additional strategy classes that implement the FilterStrategy interface. This supports the Open/Closed Principle, since the system can be extended with new filtering behaviors without modifying existing code.

Overall, the Strategy Pattern provides a scalable solution for handling flexible event search functionality while keeping the service layer organized and maintainable.

Design Pattern 2: Observer Pattern

The Observer Pattern is used in SunDevil Connect to implement the system's notification functionality. In this design, the system acts as a Subject that maintains a list of observers, while client-side components act as Observers that respond when a notification is generated. Examples of events that trigger notifications include:

- Approval or rejection of club membership requests
- Confirmation of event registrations
- Posting of new club announcements
- Updates to events that users have registered for

The system includes a NotificationService that maintains a mapping between user identifiers and their associated observers. When a user logs into the system, observers associated with that user session are attached to the notification service. When the user logs out, these observers are removed.

When a system event occurs that requires notification, the corresponding controller (such as the event registration controller or announcement controller) creates a Notification object and sends it to the NotificationService. The service then identifies the affected users and notifies their observers. Each observer receives the update and determines how the notification should be displayed within the user interface.

This design decouples the components responsible for generating events from those responsible for presenting notifications. Controllers and services only need to generate notification objects and pass them to the notification service, without needing to understand how the user interface handles them.

The Observer Pattern therefore improves modularity, scalability, and separation of concerns within the system architecture.

Design Pattern 3: Singleton Pattern

The Singleton Pattern is used in SunDevil Connect to ensure that certain core infrastructure services maintain a single shared instance across the system. These services provide centralized functionality that must remain consistent throughout the application.

Examples of Singleton services include:

- AuthenticationService
- DatabaseService

- NotificationService

The AuthenticationService manages user authentication, session validation, and logout functionality. Ensuring that only one instance of this service exists prevents inconsistent session states and guarantees that all controllers reference the same authentication source of truth. Similarly, the DatabaseService acts as the centralized interface for database operations, including saving entities, retrieving records, updating data, and executing queries. Maintaining a single instance helps avoid conflicting database operations and unnecessary resource usage.

The NotificationService is also implemented as a Singleton because it maintains the global registry of observers associated with active user sessions. Having multiple instances of this service could lead to fragmented observer lists and inconsistent notification delivery.

By enforcing a single instance of these core services, the Singleton Pattern promotes architectural stability, consistent system behavior, and efficient resource management across the entire application.

Note: If you have additional design patterns identified, please list them as well using the same format above

Design Pattern 4: Factory Pattern

The Factory Pattern is used in SunDevil Connect to dynamically create the appropriate filter strategies based on the filter parameters selected by the user. Because each filtering strategy requires different configuration data (for example, a date filter requires a date range while a category filter requires a category value), strategies cannot be instantiated interchangeably without additional construction logic.

To address this, the system introduces a FilterStrategyFactory responsible for creating the appropriate strategy objects based on the provided filter parameters. The factory receives a collection of filter inputs, represented as a mapping between FilterType values and their associated parameters, and constructs the corresponding filtering strategies. For example, if a user selects filters for date range and category, the factory will create both a DateFilterStrategy and a CategoryFilterStrategy. These strategy objects are returned as a list and applied sequentially to the event dataset.

By delegating strategy construction to a factory, the event browsing controller does not need to know how individual strategies are instantiated. Instead, it simply requests the necessary strategies from the factory and applies them to the list of events.

This design improves maintainability by centralizing object creation and separating it from business logic. If additional filtering criteria are introduced in the future, the factory can be extended to support the new strategy types without modifying existing components. The Factory Pattern therefore complements the Strategy Pattern by enabling flexible and scalable creation of filtering behaviors.

1. Refined class diagram clearly shows the design pattern related to classes and other classes. Astah must be used to develop the class diagram. (Copy and paste your class diagram here. Make sure to use the correct UML notations. Points will be given for the completeness and the correctness of the diagram)



